

Formal Analysis of Human Forgetting in Security Ceremonies

Anonymous authors

Abstract

Security ceremonies extend security protocols by incorporating human actions. Consequently, their security depends not only on the underlying cryptographic mechanisms but also on how humans handle and exchange information. Humans may deviate from the expected behavior during a ceremony. The X-Men approach introduced and formalized the idea of mutations to model such deviations and to analyze how human mistakes can compromise the overall security of the ceremony.

In this work, we extend the X-Men approach with a new mutation, the *forget* mutation, which captures cases in which a human user forgets information during a ceremony. Specifically, we formalize this mutation by means of an algorithm that captures two distinct cases: (1) a user may forget to add some newly acquired information to their knowledge (e.g., a newly issued key in an authentication ceremony), or (2) a user may forget some information they previously possessed (e.g., a password in a user identification ceremony). The *forget* mutation is a primitive mutation in the sense that it cannot be expressed in terms of existing mutations, and thus allows us to identify sources of novel vulnerabilities.

We have implemented the *forget* mutation in X-Men 2.0, a prototypical extension of the X-Men tool, and applied it to a Bank Login ceremony as a proof-of-concept. Our analysis identifies vulnerabilities that arise from human memory lapses rather than from weaknesses in the underlying cryptographic protocol.

CCS Concepts

• **Security and privacy** → **Formal methods and theory of security; Formal security models; Logic and verification; Human and societal aspects of security and privacy; Usability in security and privacy;**

Keywords

Security ceremonies, Mutations, Formal methods, Modeling and analysis, User authentication

ACM Reference Format:

Anonymous authors. 2026. Formal Analysis of Human Forgetting in Security Ceremonies. In *Proceedings of 18th International Conference on Advanced Visual Interfaces (AVI'26)*. ACM, New York, NY, USA, 12 pages. <https://doi.org/XXXXXXX.XXXXXXX>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

AVI'26, Venice, Italy

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
<https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Context. A security ceremony extends the concept of a security protocol by explicitly including human users who exchange messages and data with computing systems, for example, in online payment, voting, or critical infrastructures [10]. Security ceremonies may suffer from vulnerabilities that stem from unexpected or incorrect human behavior during interactions with other ceremony participants (or agents), such as the system, or with attackers that are internal or external to the system. For example, in a user authentication ceremony, a user may inadvertently disclose credentials to an adversarial interface as a result of deception [3], which can compromise security even when the underlying cryptographic protocol remains correct. Users may also confuse which credential (e.g., a password) corresponds to which service, leading them to enter valid passwords but associated with other services [12].

Several formal approaches (e.g., [3, 5, 6, 11, 21]) have been proposed to verify that security ceremonies satisfy their intended security goals. Formal methods, including formal tools, can model and analyze erroneous human interactions and behaviors, even in the absence of attacks. In this paper, we build on the X-Men approach by Sempredoni and Viganò [22], which models human behaviors via mutation rules, generates mutations in the behavior of the other ceremony participants to match the human-induced mutations, and propagates these mutations throughout the ceremony, enabling analysis of both the original ceremony specification and its mutated variants. To automate this approach, they developed the X-MEN tool [21, 22], which is built on top of Tamarin Prover [2, 14]. To date, X-Men supports the *skip*, *add*, *replace*, and *neglect* mutations, as well as the combined *disorder* mutation [16].

Contributions. The main contribution of this paper is extending the X-Men approach with a new human mutation, *forget*, which cannot be expressed using existing mutations. We have also implemented X-Men 2.0, a prototypical extension of the X-Men tool with a modular architecture that integrates *forget* and supports the future integration of other new mutations.

The *forget* mutation formally models the fact that human agents participating in a ceremony may forget some important information, such as a newly issued key, or a password in a user authentication ceremony. Consequently, they may attempt to complete the ceremony by using instead other passwords or other information that they still remember, thereby possibly endangering the security of the ceremony (this is discussed in detail in Section 4).

We formalize the *forget* mutation by means of an algorithm that restricts an agent's knowledge so that the forgotten information is no longer accessible. Forgetting does not erase knowledge, but limits which previously known messages can be used in subsequent ceremony steps. To apply the mutation to an agent's knowledge, given a message m that an agent intends to derive and send, we check whether any messages used in a derivation of m are contained in the agent's set of forgotten messages (see Section 3). We also formalize and analyze how the effects of the *forget* mutation propagate through subsequent ceremony states (see Section 5).

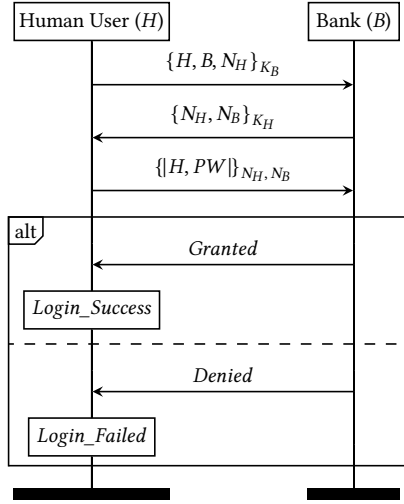


Figure 1: Sequence diagram of the Bank Login ceremony.

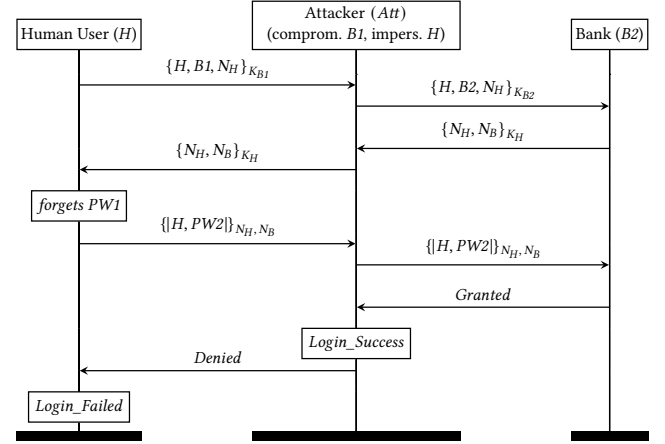
The *forget* mutation algorithm is generic and independent of any specific formal verification tool. It can therefore be applied to any formal approach for analyzing security ceremonies with human mistakes. For concreteness, we provided a prototypical implementation of the *forget* mutation in X-Men 2.0, and we applied it to a Bank Login ceremony as a proof-of-concept (see Section 2). Related work is discussed in Section 6, and concluding remarks are given in Section 7. An appendix, which is provided as supplemental material, gives the full details of our formalization.

2 Case Study: A Bank Login Ceremony

Scenario Description. Figure 1 shows a simple but illustrative e-banking login ceremony in which a human user H interacts with a bank B to gain access to their account. One security goal (among others) of this ceremony is to ensure that both ceremony participants, H and B , mutually authenticate and that access to the bank account is granted only to a legitimate user who provides the correct credentials, given by the username H and a password PW .

For simplicity, we have based the ceremony on a variant of the Needham-Schroeder Public-Key (NSPK) protocol [15], but, as we will see, the attack that we focus on is not due to the well-known authentication flaw of this protocol but rather on the reuse of a password that belongs to a different account. H initiates the exchange with a message encrypted with B 's public key K_B , containing the human's identifier H , the bank's identifier B , and a nonce N_H .¹ B replies with N_H and its own nonce N_B , encrypted with H 's public key K_H . Finally, to provide their credentials, H demonstrates knowledge of the password PW by sending it in encrypted form using a symmetric key obtained by pairing the two nonces. If B verifies PW against its stored credentials, the login succeeds and a message *Granted* is sent; otherwise, *Denied* is sent.

¹A *nonce* is a number generated for one-time use in cryptographic communications to prevent replay attacks. Note that, as is standard, we write, e.g., N_X to represent graphically that this freshly-generated identifier was generated by X , but there is nothing in N_X that reveals to other agents that it was indeed generated by X .


 Figure 2: Ceremony under attack with the *forget* mutation.

The Ceremony under Attack. Suppose that a human user H holds two accounts, one at bank $B1$ and another at bank $B2$, both with the same user identifier H , but with different passwords $PW1$ and $PW2$, respectively. Then, H could be attacked as shown in Figure 2. H starts a ceremony with bank $B1$ to get access to that account. We assume that Att , a standard Dolev-Yao attacker [9], has compromised $B1$, obtaining its private key. Att can thus read all messages sent by H to $B1$, and it can carry out a parallel execution of the ceremony with $B2$, impersonating H . Note that impersonation does not mean nor entail compromise: Att pretends to be H in the message to $B2$, but does not have H 's private key. Hence, Att cannot decrypt $B2$'s response $\{N_H, N_B\}_{K_H}$, but that is also a valid response for H 's ceremony with $B1$ so Att simply forwards it.

H has, however, forgotten the password $PW1$ for $B1$. Instead of aborting the session, H , as humans often do when they forget a password [12], tries using another password they recall, $PW2$, thinking it might be the right password. The message containing $PW2$ is encrypted using the symmetric key obtained by pairing the nonces N_H and N_B . Again, Att cannot read the message contents (as they do not know N_B), and thus does not learn the password, but that is irrelevant since $B2$ accepts it as a valid message and gives Att access to H 's account at $B2$. Finally, Att may choose whether to conclude the ceremony between H and $B1$; in Figure 2, we show the case in which H is informed that the login failed. In any case, Att can now drain not only H 's account at $B1$ (which Att had already compromised), but also H 's account at $B2$.

As we remarked, the NSPK protocol has a number of well-known weaknesses, but we chose it as we were looking for a simple way to illustrate the power of the *forget* mutation. In particular, we adapted this NSPK-style message flow because it establishes a fresh challenge-response exchange based on two nonces, which are then used as a symmetric key to protect the password in the final message. As a result, the attacker Att never learns $PW2$ as they do not know N_B and cannot decrypt messages encrypted using the key derived using N_B . Nevertheless, the security goal still fails, and, notably, this attack does not rely on password disclosure or any weakness in the cryptography. This violation arises solely from a human mistake:

forgetting a password and providing a valid password for the wrong service. This is precisely the behavioral deviation that the *forget* mutation allows us to capture. This shows that a security breach may occur even when long-term secrets remain confidential, and the cryptographic protocol behaves correctly.

3 Preliminaries

In security ceremonies, interactions between participating agents are modeled as the exchange of structured messages that convey information relevant to the ceremony, such as their identities, actions, and goals (e.g., to share a secret or authenticate other agents). In this section, we follow standard terminology to formally define the message algebra, give a threat model over the knowledge possessed by each agent (which determines their ability to interpret, construct, and/or transform messages), specify a security ceremony, and define an execution model and security goals.

3.1 Algebra of Messages

Definition 3.1. The *algebra of messages* $T_\Sigma(\mathcal{V})$ consists of terms that model the messages exchanged between the agents, where Σ is a signature and $\mathcal{V}$ is a countably infinite set of variables. Σ contains three pairwise disjoint sets: C_{Fresh} , C_{pub} , and $\mathcal{F}$. C_{Fresh} is a countably infinite set of fresh constants, modeling nonce generation. C_{pub} is a countably infinite set of public constants, representing agent names and other publicly known values. $\mathcal{F}$ is a finite set of function symbols, representing various message operations: the *pairing* $pair(m_1, m_2)$ of two messages m_1 and m_2 , which we also denote by $\langle m_1, m_2 \rangle$; the *symmetric encryption* $senc(m_1, m_2)$ or $\llbracket m_1 \rrbracket_{m_2}$ of a message m_1 with a message m_2 and the *symmetric decryption* $sdec(m_1, m_2)$ of m_1 with m_2 ; the *asymmetric encryption* $aenc(m_1, m_2)$ or $\{m_1\}_{m_2}$ of a message m_1 with a message m_2 and the *asymmetric decryption* $adec(m_1, m_2)$ of m_1 with m_2 . Other operators (e.g., for digital signatures, hashing, or modular exponentiation) can be added as needed and are handled in the standard way.

For readability, we also use the variable k to denote symmetric/asymmetric keys and k^{-1} to denote inverse/private asymmetric keys, and write k for $(k^{-1})^{-1}$. For brevity, we also write, e.g., $\langle m_1, m_2, m_3 \rangle$ for $\langle m_1, \langle m_2, m_3 \rangle \rangle$ when there is no risk of confusion for the analysis of the pair. We might also write m_1, m_2 for $\langle m_1, m_2 \rangle$.

Definition 3.2. We say that m_1 is a *submessage* of m_2 , written $m_1 \in submsg(m_2)$, iff: $m_2 = m_1$; $m_2 = \langle m_3, m_4 \rangle$ for some m_3 and m_4 , and $m_1 \in submsg(m_3)$ or $m_1 \in submsg(m_4)$; $m_2 = h(m_3)$ for some m_3 and $m_1 \in submsg(m_3)$; $m_2 = \circ(m_3, k)$ for some m_3 and k , with $\circ \in \{senc, aenc\}$ and $m_1 \in submsg(m_3)$. A message m is *atomic* if it has no proper submessages other than itself.

Definition 3.3. The *format* $f = format(m)$ of a message m is its top-level function symbol in $\mathcal{F}$: if m has no top-level function symbol, then $f = m$; if $m = \langle m_1, m_2 \rangle$ for some m_1 and m_2 , then $f = pair$; if $m = h(m_1)$ for some m_1 , then $f = h$; if m is $\circ(m_1, k)$ for some m_1 and k , with $\circ \in \{senc, aenc\}$, then $f = \circ$.

3.2 Threat Model: Knowledge Inference Rules

Tamarin, and therefore X-Men, come equipped with the standard *Dolev-Yao (DY) attacker model* [9], in which the attacker controls

the network but cannot break cryptography.² In particular, the attacker can intercept messages, can analyze encrypted messages if they possess the corresponding decryption keys, and can generate messages and send them under any agent's name. For instance, the presence of $\{m_1, m_2\}_k$ in an agent's (legitimate or attacker) knowledge does not imply that they can derive m_1 or m_2 , unless they also possess the corresponding decryption key k^{-1} .

Formally, an agent's knowledge $\mathcal{K}$ consists of the messages that the agent can derive using the following DY-style inference rules:

$$\begin{array}{ll} \frac{m_1 \ m_2}{m_1, m_2} \text{ (Pair generation)} & \frac{m_1, m_2}{m_i} \text{ (Pair analysis), for } i \in \{1, 2\} \\ \frac{m \ k}{\llbracket m \rrbracket_k} \text{ (Symm. encryption)} & \frac{\llbracket m \rrbracket_k \ k}{m} \text{ (Symm. decryption)} \\ \frac{m \ k}{\{m\}_k} \text{ (Asymm. encryption)} & \frac{\{m\}_k \ k^{-1}}{m} \text{ (Asymm. decryption)} \end{array}$$

A *derivation* is a finite sequence of applications of DY inference rules that derives a target message from a given set of hypotheses, typically an agent's knowledge. Such derivations are naturally represented as *derivation trees*, where nodes correspond to messages and edges correspond to rule applications [17]. We write $\mathcal{K}_H^i \vdash m$ to express that in state i , the human agent H can derive the message m from their current knowledge (i.e., set of messages) $\mathcal{K}_H^i$.

Following standard practice in symbolic protocol analysis (see, e.g., [18]), we restrict our attention to *normal derivations*. In a normal derivation, no rule application is immediately undone by its inverse, i.e., pair generation is not immediately followed by pair analysis, and an encryption is not immediately followed by its decryption, nor vice versa. Then, any derivation containing such cyclical fragments can be normalized by removing them, without changing the derivable message. This restriction still accounts for all possible derivations: if a message is derivable, then it admits a normal derivation [7, 18]. So, to determine if a message is derivable or not, we only need to search the finite space of normal derivations.

3.3 Ceremony Specification

Formal modeling of a security ceremony specifies agents' behavior as they execute *roles*, i.e., sequences of events such as sending or receiving messages, generating fresh values, and initializing executions. We follow Tamarin's syntax and adopt its core syntactic elements for ceremony specification. A *role-script* formalizes a role as a sequence of *events* e , each executing exactly one *action* $a \in RoleActions = \{Snd, Rcv, Fresh, Start\}$. Send and receive events take the form $Snd(A, l, P, m)$ and $Rcv(A, l, P, m)$, where A denotes the executing role, $l \in ChannelType = \{insec, conf, auth, sec\}$ specifies the channel type (insecure, confidential, authentic, secure, respectively), P denotes the intended communication partner, and m is the transmitted message. For send events, P denotes the intended recipient; for receive events, it denotes the apparent sender (the attacker may have forged the identity). The event $Fresh(A, m)$ denotes the generation of a fresh value (e.g., a nonce or key) by role

²Note, however, that X-Men's approach also allows one not to consider the presence of an attacker, and instead focus on capturing how human agents might interact insecurely with other ceremony agents even in the absence of an attacker. Thus, all non-human agents behave honestly and follow the steps of the ceremony, while the human(s) may still make mistakes.

RoleScript_H =
 $[Start(H, \langle \langle 'agentname', 'agentname', 'password', 'key', 'key', 'key' \rangle, \langle H, B, PW, K_H, K_H^{-1}, K_B \rangle \rangle),$
 $Snd(H, conf, B, \langle \langle 'agentname', 'agentname', 'nonce' \rangle, \langle H, B, N_H \rangle \rangle_{K_B})$
 $Rcv(H, conf, B, \langle \langle 'nonce', 'nonce' \rangle, \langle N_H, N_B \rangle \rangle_{K_H}),$
 $Snd(H, sec, B, \langle \langle 'agentname', 'password' \rangle, \langle H, PW \rangle \rangle_{N_H, N_B}),$
 $Rcv(H, inse, B, \langle 'access', 'Granted' \rangle)]$

Figure 3: Role-script for H in the Bank Login ceremony.

A. The $Start(A, \mathcal{K})$ event initializes a role-script with the initial knowledge $\mathcal{K}$ of A and occurs once per ceremony execution.

We use Tamarin *constants* to label values exchanged between roles or stored in their knowledge. Labels are written in quotes; for example, 'password' labels PW , $PW1$, and $PW2$. They restrict the scope of mutations to preserve type safety, ensuring values can only be replaced by others with the same label. Accordingly, messages follow a constant-value structure $\langle l, m \rangle$.

The Bank Login ceremony consists of two roles: the human user H and the bank B (Figure 1). The role-script for H is shown in Figure 3, while the role-script for B appears in Figure 5 in the appendix (provided as supplemental material).

3.4 Execution Model

We rely on Tamarin's execution model, where protocols are specified using *multiset rewriting rules* to construct a Labeled Transition System that captures the evolution of the system's state based on the ceremony's actions and the agents' knowledge and behavior.

A *state* is a multiset of *facts*, where *linear facts* model consumable resources and can be added to and removed from the system state, whereas *persistent facts* (prefixed with "!") model non-consumable resources, which, once added, can never be removed from the system state. The *initial state* is the empty multiset $[]$. A *trace* tr is a finite sequence of multisets of actions a , generated by applying labelled state transition rules of the form

$$prem \xrightarrow{a} conc \quad (1)$$

Such a rule can be applied to a state that contains facts matching the premise $prem$. This removes the matching linear facts from the state, adds the instantiated facts from the conclusion $conc$ to the state, records the instantiated actions from a in the trace, and moves the system to a new state.

An example of an excerpt of a trace of actions in the Bank Login ceremony involving a human H and bank B is as follows:

$$\begin{aligned} &Snd(H, conf, B, \langle H, B, N_H \rangle_{K_B}) \\ &Rcv(B, conf, H, \langle N_H, N_B \rangle_{K_H}) \\ &Snd(H, sec, B, \langle \langle H, PW \rangle \rangle_{N_H, N_B}) \end{aligned}$$

A ceremony model comprises *agent rules* specifying the agents' state transitions and communication. For each event e in agent A 's role-script, the transition rule in Equation (1) is defined as follows:

- $prem$ contains an agent state fact $AgSt(A, step, \mathcal{K}_A^{step})$ and $conc$ contains the subsequent agent state fact $AgSt(A, step, \mathcal{K}_A^{step'})$, where $step$ refers to the role step the agent is in and $\mathcal{K}_A^{step}$ is the agent A 's knowledge at that step;
- if event e is $Start(A, m)$, then it is translated to a setup rule, where $conc$ contains agent's initial state $AgSt(A, 0, m)$;

(H_0) $[]$
 $\xrightarrow{Start(H, \langle \langle 'agentname', 'agentname', 'password', 'key', 'key', 'key' \rangle, \langle H, B, PW, K_H, K_H^{-1}, K_B \rangle \rangle)}$
 $[AgSt(H, 1, \langle H, B, PW, K_H, K_H^{-1}, K_B \rangle)]$
 (H_1) $[AgSt(H, 1, \langle H, B, PW, K_H, K_H^{-1}, K_B \rangle)]$
 $\xrightarrow{Snd(H, conf, B, \langle \langle 'agentname', 'agentname', 'nonce' \rangle, \langle H, B, N_H \rangle \rangle_{K_B})}$
 $[AgSt(H, 2, \langle H, B, PW, K_H, K_H^{-1}, K_B, N_H \rangle), Out_{conf}(H, B, \langle \langle 'agentname', 'agentname', 'nonce' \rangle, \langle H, B, N_H \rangle \rangle_{K_B})]$

Figure 4: Some of the agent rules for H .

- if e is $Snd(A, l, P, m)$, then $conc$ additionally contains an outgoing message fact $Out_l(A, P, m)$;
- if e is $Rcv(A, l, P, m)$, then $prem$ contains an incoming message fact $In_l(P, A, m)$.

Thus, we can define an example execution trace as follows:

$$\begin{aligned} &\vdots \Sigma_1 \\ &AgSt(H, i, \mathcal{K}_H^i), Pre_i, Rcv(H, l_1, A_1, m_1) \rightarrow \\ &AgSt(H, i+1, \mathcal{K}_H^{i+1}), Post_{i+1}, Snd(H, l_2, A_2, m_2) \\ &\vdots \Sigma_2 \\ &AgSt(H, i+1, \mathcal{K}_H^{i+1}), Pre_{i+1}, Rcv(H, l_3, A_3, m_3) \rightarrow \\ &AgSt(H, i+2, \mathcal{K}_H^{i+2}), Post_{i+2}, Snd(H, l_4, A_4, m_4) \\ &\vdots \Sigma_3 \end{aligned} \quad (2)$$

At the beginning of a ceremony execution, each agent A has an initial knowledge $\mathcal{K}_A^0$. At each execution step, A 's knowledge $\mathcal{K}_A^{step}$ is closed under the inference rules defined in Section 3.2, which are applied to $\mathcal{K}_A^0$ extended with all the messages that A has received so far, unless A forgets something, as discussed in Section 4.

An excerpt of the agent rules for the human user H in the Bank Login ceremony is shown in Figure 4; the complete rules, including those for the bank B , appear in Figures 6 and 7 in the appendix (provided as supplemental material).

3.5 Security Goals

Security goals express the security properties a ceremony should guarantee. In our setting, we "pushed" security from the system to the human users, as human errors during interaction can jeopardize security. We show this with the following security goals that we analyze for the Bank Login ceremony (cf. Section 2):

- [SG1] Only a legitimate account holder should be able to authenticate with the bank to obtain access to their account.
- [SG2] No illegitimate user (e.g., an attacker) should be able to learn the human's password, even during an attack.

3.6 Modeling Human Mutations

Human mutations during a ceremony execution can affect the behavior of other ceremony agents and, thus, the whole ceremony. Hence, we define (1) human mutations that modify a ceremony trace by mutating the sub-trace of a human agent, and (2) the mutations on the sub-trace(s) of other agent(s) in the ceremony that are required to match the human mutations. According to the ceremony specification, algorithms for matching mutations adapt the send/receive patterns of other agents to align with the human mutation and propagate these changes across all roles of the ceremony.

This ensures that the ceremony execution can continue beyond the mutation point and produces a fully executable mutated ceremony model that can terminate. This model is then provided to Tamarin for execution and security analysis with respect to the ceremony's security goals (potential false positives may require validation by ceremony designers) [22].

Formally, a generic *human mutation* is a function $\mu^H : tr \rightarrow tr'$ that takes as input a trace tr and gives as output a new trace $tr' = \llbracket tr \rrbracket^{\mu^H}$ obtained by mutating H 's sub-trace due to the human H "deviating" from the original role-script. A *matching mutation* μ^m for a human mutation modifies the sub-traces of other agents to "match" and "propagate" the human mutation μ^H . The combination $\mu^H \circ \mu^m : tr \rightarrow tr'$ of the two mutations takes as input a trace tr and gives as output a new trace $tr' = \llbracket tr \rrbracket^{\mu^H \circ \mu^m}$ in which the human mutation is matched and propagated. We denote the sub-trace of a human agent H in a ceremony execution as $[a_0, \dots, a_i, \dots, a_n]^H$, where $0 < i \leq n$, and $\llbracket \cdot \rrbracket^\mu$ applies not just to traces but also indirectly applies to the role-scripts and Tamarin actions.

4 The Forget Mutation

The *forget* mutation captures a human-specific behavior, i.e., the loss of access to previously known information. This mutation differs from the existing mutations implemented in X-Men [21, 22]: *skipping* one or more of the actions that the ceremony expects the human user to carry out (such as sending or receiving a message), *adding* an action (e.g., sending a message twice), *replacing* a message with another one (e.g., sending a sub-message of the original message), *neglecting* to adhere to one or more internal behaviors expected by the ceremony (such as neglecting to check the contents of a message), changing the order of actions (via the *disorder* mutation). These operate on the actions performed by the human agents or the messages they send. In contrast, the *forget* mutation affects the human agent's knowledge, restricting which information remains available for them to compose or decompose messages, which may affect the actions they perform, as we will see in Example 4.6 and Algorithm 1.³

4.1 Defining the Forget Mutation

To formalize *forget*, we use the mutation structure in Section 3.6.

Definition 4.1. The *forget* mutation $\mu_{\text{forget}}^H : tr \rightarrow tr'$ is a human mutation applied to a human sub-trace $[a_0, \dots, a_i, \dots, a_n]^H$ of a trace tr such that it modifies the knowledge $\mathcal{K}_H^i$ at state i of a human agent H participating in the ceremony either (1) by causing H to forget to add to $\mathcal{K}_H^i$ a message m that H has just received, or (2) by causing H to forget a message m already in $\mathcal{K}_H^i$, preventing the forgotten m to be used when deriving other messages from $\mathcal{K}_H^i$. To capture the forgotten messages in case (2), we introduce a forget set Forget_H^i , which contains messages that are no longer accessible to H for message derivation because H has forgotten them.

To determine whether a derivation π of a message m from the agent's current knowledge $\mathcal{K}_H^i$ is affected by the corresponding

forget set Forget_H^i , we check whether π depends on a forgotten message $\bar{m}$, i.e., whether $\bar{m} \in \text{Forget}_H^i$ occurs as a hypothesis in the derivation. If so, then the derivation is not allowed.

We write $\mathcal{K}_H^i \vdash_{\text{Forget}_H^i} m$ to express that m is derivable from $\mathcal{K}_H^i$ without using any hypothesis contained in the forget set Forget_H^i . We will formalize $\mathcal{K}_H^i \vdash_{\text{Forget}_H^i} m$ in Definition 4.5. Before that, we illustrate these notions with a few examples, highlighting different options we can consider.

Example 4.2 (Different occurrences of a forgotten message in a derivation). Note that Definition 4.1 does not mean that a message $\bar{m} \in \text{Forget}_H^i$ is not allowed to occur in a derivation; we only exclude the hypotheses of the derivation. For instance, if

$$\mathcal{K}_H^i = \{ \llbracket m \rrbracket_{\bar{m}}, \bar{m} \} \text{ and } \text{Forget}_H^i = \{ \bar{m} \}$$

then $\mathcal{K}_H^i \vdash m$ as follows

$$\frac{\llbracket m \rrbracket_{\bar{m}} \quad \bar{m}}{m} \text{ (Symm. decryption)}$$

but $\mathcal{K}_H^i \not\vdash_{\text{Forget}_H^i} m$ since $\bar{m}$ is a hypothesis in the derivation.

If

$$\mathcal{K}_H^i = \{ \llbracket m \rrbracket_{\bar{m}}, \llbracket \bar{m} \rrbracket_{m_1}, m_1 \} \text{ and } \text{Forget}_H^i = \{ \bar{m} \}$$

then $\mathcal{K}_H^i \vdash m$ as follows

$$\frac{\llbracket m \rrbracket_{\bar{m}} \quad \frac{\llbracket \bar{m} \rrbracket_{m_1} \quad m_1}{\bar{m}} \text{ (Symm. decryption)}}{m} \text{ (Symm. decryption)}$$

and also $\mathcal{K}_H^i \vdash_{\text{Forget}_H^i} m$ since $\bar{m}$ is not a hypothesis but simply occurs in the derivation.

We now give examples of the two ways H can forget a message.

Example 4.3 (Forgetting a message before adding it to the knowledge). Consider the execution trace in Equation 2 and let the human agent H possess the knowledge $\mathcal{K}_H^i = \{ m_2, m_4 \}$ at state i . If H , in transition i , receives m_1 from another agent A_1 , but forgets to add it to their knowledge, then the knowledge is not updated and m_1 is not added to the forget set:

$$\mathcal{K}_H^{i+1} = \mathcal{K}_H^i = \{ m_2, m_4 \} \text{ and } \text{Forget}_H^{i+1} = \text{Forget}_H^i.$$

Notably, other instances of m_1 remain intact in $\mathcal{K}_H^i$, provided H had previously known them.

Example 4.4 (Forgetting a message that is already in the knowledge). Consider the execution trace in Equation 2 and let the human agent H possess the knowledge $\mathcal{K}_H^i = \{ \llbracket m_2, m_4 \rrbracket_{m_5}, m_2, m_4 \}$. Suppose that, in transition i , H receives a message m_1 and forgets m_2 . Since the knowledge is monotonically increasing, m_2 remains in $\mathcal{K}_H^{i+1}$ but it is added to the forget set:

$$\mathcal{K}_H^{i+1} = \mathcal{K}_H^i \cup \{ m_1 \} = \{ \llbracket m_2, m_4 \rrbracket_{m_5}, m_2, m_4, m_1 \}$$

$$\text{Forget}_H^{i+1} = \text{Forget}_H^i \cup \{ m_2 \}.$$

Once m_2 is in Forget_H^{i+1} , H cannot use it to derive any message. If at a later time $j \geq i + 1$, H receives m_2 again in a transition, say $\text{Rcv}(H, l, A, m_2)$ or some other more complex message that allows H to derive m_2 , then the forget set updates to

$$\text{Forget}_H^{j+1} = \text{Forget}_H^{i+1} \setminus \{ m_2 \},$$

and m_2 can again be used in derivations.

³This is why *forget* is a primitive mutation that cannot be expressed in terms of existing mutations. The first case of the *forget* mutation in Algorithm 1, forgetting the received message, can be expressed via skipping the receive action, but the main case, forgetting a message that is already in the human's knowledge, cannot. And it is that case that gives rise to the attack that we describe in this paper.

4.2 Blocked Derivations

Our definitions allow us to consider restrictions imposed by the forget set of varying strength. In particular, we distinguish three cases, depending on whether the hypothesis $\bar{m}$ of π is explicitly included in Forget_H^i or can be inferred from the messages in Forget_H^i :

$$\text{hypothesis } \bar{m} \text{ of } \pi \text{ is s.t. } \begin{cases} \bar{m} \in \text{Forget}_H^i & \text{(Case 1)} \\ \text{Forget}_H^i \vdash \bar{m} \text{ using DY rules for pairs} & \text{(Case 2)} \\ \text{Forget}_H^i \vdash \bar{m} \text{ using all DY rules} & \text{(Case 3)} \end{cases}$$

These conditions are independent of the *forget* mutation algorithm itself. Algorithm 1 below works uniformly for all three cases; the chosen case only determines which derivation hypotheses are treated as *blocked* by forgetting.

Formally, let Hyp_π denote the set of hypotheses of a derivation π of m from $\mathcal{K}_H^i$. We define the blocking function as:

$$\text{Blocked}(\text{Hyp}_\pi, \text{Forget}_H^i) = \begin{cases} \text{Hyp}_\pi \cap \text{Forget}_H^i & \text{(Case 1)} \\ \{ \bar{m} \in \text{Hyp}_\pi \mid \text{Forget}_H^i \vdash_{\text{pairing_rules}} \bar{m} \} & \text{(Case 2)} \\ \{ \bar{m} \in \text{Hyp}_\pi \mid \text{Forget}_H^i \vdash \bar{m} \} & \text{(Case 3)} \end{cases}$$

Definition 4.5. We say that m is *derivable under forgetting* at state i , and write $\mathcal{K}_H^i \vdash_{\text{Forget}_H^i} m$, if there exists at least one derivation π of m from $\mathcal{K}_H^i$, in symbols $\mathcal{K}_H^i \vdash m$, whose hypotheses are not blocked:

$$\mathcal{K}_H^i \vdash_{\text{Forget}_H^i} m \quad \text{iff} \quad \text{exists } \pi \text{ of } m \text{ from } \mathcal{K}_H^i \text{ such that} \\ \text{Blocked}(\text{Hyp}_\pi, \text{Forget}_H^i) = \emptyset.$$

The definition of the *forget* mutation is parameterized by the three possible cases for the “closure” of the forget set, each formalizing a different “ability” of the forgetting human. In *Case 1*, forgotten messages are treated as atomic and non-decomposable: if a composite message (e.g., a pair) is forgotten, it is excluded as a whole, although its components may still be used as hypotheses if they appear independently in the agent’s knowledge. This resembles settings where credentials are stored or remembered as indivisible records. *Case 2* makes forgetting transparent with respect to pairing, closing the forget set under pair composition and decomposition. Forgetting a pair entails forgetting its components, and forgetting both components entails forgetting the pair. This reflects a human’s ability to decompose and recombine structured information and represents a balanced assumption between expressiveness and tractability. *Case 3* is the most general: the forget set is closed under all DY rules, meaning that any message constructible from forgotten elements is itself considered forgotten. This models derivations that depend on multiple components, such as multi-step authentication, where forgetting any required part blocks the entire process. While expressive, this case may overly restrict derivability and is not required for the analysis we considered.

In our analysis, we adopt *Case 1*, which fits the Bank Login ceremony, where the user identifier and password are sent separately (in different steps) rather than as a pair, making the non-decomposable interpretation applicable. Depending on the ceremony’s structure and requirements, any of the three cases may be chosen; for example, if identifier and password are transmitted together as a pair (in the same step), *Case 2* may model forgetting more accurately.

Example 4.6 (Derivability under Forgetting). In the following three examples, we consider m_1 as the message to be derived.

- (1) $\mathcal{K}_H^i = \{ m_1, \{m_1\}_{m_2}, m_2 \}$ and $\text{Forget}_H^i = \{ \{m_1\}_{m_2} \}$.

In *Case 1*, there are two derivations of m_1 : one direct, using $m_1 \in \mathcal{K}_H^i$, and another via decryption of $\{m_1\}_{m_2}$. The second derivation is forbidden because it relies on a forgotten message, but the first remains valid. In *Case 2* and *Case 3*, the reasoning is similar: forgetting $\{m_1\}_{m_2}$ blocks all derivations that depend on it, yet m_1 remains derivable via its direct occurrence in the knowledge. Thus, forgetting restricts the set of valid derivations but does not prevent derivability when alternative derivations exist.

- (2) $\mathcal{K}_H^i = \{ m_1, \{m_1\}_{m_2}, m_2 \}$ and $\text{Forget}_H^i = \{ \{m_1\}_{m_2}, m_2 \}$.

In *Case 1*, m_1 can still be derived directly since it is explicitly present in $\mathcal{K}_H^i$, even though both $\{m_1\}_{m_2}$ and m_2 are forgotten. In *Case 2* and *Case 3*, all derivations that use messages that can be derived from Forget_H^i are forbidden. In particular, deriving m_1 by decrypting $\{m_1\}_{m_2}$ would require using m_2 , which is in Forget_H^i . Therefore, this derivation is invalid (i.e., blocked), and m_1 cannot be derived under the constraint of the forget set in *Case 3*.

- (3) $\mathcal{K}_H^i = \{ \langle m_1, m_2 \rangle, m_1, m_2 \}$ and $\text{Forget}_H^i = \{ \langle m_1, m_2 \rangle \}$.

In *Case 1*, $\langle m_1, m_2 \rangle$ is treated as non-decomposable; however, since m_1 and m_2 are individually known, they remain derivable. In *Case 2* and *Case 3*, decomposition rules allow extracting m_1 and m_2 from the pair in Forget_H^i , so no derivation of m_1 and m_2 is valid as any such derivation would rely on a message derivable from Forget_H^i . Hence, despite m_1 and m_2 existing individually in the knowledge, they become usable.

4.3 The Algorithm of the Forget Mutation

The *forget* mutation is formalized by Algorithm 1. When the human agent forgets information at a transition, the algorithm updates the agent’s knowledge and forget set (Definition 4.1), and then checks whether the outgoing message (e.g., m_2) remains derivable under forgetting. If a derivation exists whose hypotheses are not blocked by the current forget set, the original send action is preserved. Otherwise, the blocked hypotheses are identified and admissible replacements are selected from the updated knowledge. Replacements must satisfy the ceremony specification and case-study constraints (e.g., message format or label compatibility; cf. Sections 3.1 and 3.3). Each admissible replacement yields a distinct mutated variant of the message, producing a separate mutated trace. If no admissible replacement exists, the send action is removed.

As human mutations must be matched and propagated to keep the ceremony executable (Section 3.6), a mutated send action is matched by mutating the corresponding receive action so that the receiver accepts the modified message. Conversely, if a send action is removed because the message cannot be constructed under the forget set Forget_H^i , the matching receive action is also removed. Propagation updates agents’ knowledge according to the messages actually received, ensuring that forgotten information remains unusable unless it is explicitly re-received.

Algorithm 1 μ_{forget}^H : Forgetting m in transition i

```

1: Consider transition  $i$ , namely  $\text{AgSt}(H, i, \mathcal{K}_H^i), \text{Pre}_i, \text{Rcv}(H, l_1, A_1, m_1) \longrightarrow \text{AgSt}(H, i + 1, \mathcal{K}_H^{i+1}), \text{Post}_{i+1}, \text{Snd}(H, l_2, A_2, m_2)$ 
2: if the forgotten message  $m = m_1$  then
3:    $[[\mathcal{K}_H^{i+1}]]^\mu := \mathcal{K}_H^i$  ▷ Forget the received message; knowledge unchanged
4: else
5:    $[[\mathcal{K}_H^{i+1}]]^\mu := \mathcal{K}_H^i \cup \{m_1\}$ 
6:    $\text{Forget}_H^{i+1} := \text{Forget}_H^i \cup \{m\}$  ▷ Add forgotten message to forget set; update knowledge with newly received message
7: end if
8:  $\Pi := \{ \pi \mid [[\mathcal{K}_H^{i+1}]]^\mu \vdash m_2 \}$  ▷ Construct all DY derivations of  $m_2$ 
9: if  $\exists \pi \in \Pi$  such that  $\text{Blocked}(\text{Hyp}_\pi, \text{Forget}_H^{i+1}) = \emptyset$  then
10:    $\text{Snd}(H, l_2, A_2, m_2)$  ▷ At least one derivation is unaffected by forgetting; send  $m_2$ 
11: else
12:    $\text{variantExists} := \text{false}$ 
13:   for each  $\pi \in \Pi$  do
14:     for each  $\bar{m} \in \text{Blocked}(\text{Hyp}_\pi, \text{Forget}_H^{i+1})$  do
15:        $R_{\bar{m}} := \{ \bar{m}' \in [[\mathcal{K}_H^{i+1}]]^\mu \mid \bar{m}' \notin \text{Forget}_H^{i+1} \text{ and } \text{format}(\bar{m}') = \text{format}(\bar{m}) \text{ and } \bar{m}' \text{ has same label as } \bar{m} \}$  ▷ Set of replacements
16:       if  $R_{\bar{m}} = \emptyset$  then
17:         continue to next  $\pi$  ▷ This derivation cannot be repaired
18:       else
19:         Generate all combinations  $C \in \prod_{\bar{m} \in \text{Blocked}(\text{Hyp}_\pi, \text{Forget}_H^{i+1})} R_{\bar{m}}$ 
20:         for each combination  $C$  do
21:           Construct variant  $[[m_2]]^\mu$  by replacing each  $\bar{m} \in \text{Blocked}(\text{Hyp}_\pi, \text{Forget}_H^{i+1})$  with the corresponding  $\bar{m}'$  from  $C$ 
22:            $\text{variantExists} := \text{true}$ 
23:           Mutate transition  $i$  to produce a new trace:
            $\text{AgSt}(H, i, \mathcal{K}_H^i), \text{Pre}_i, \text{Rcv}(H, l_1, A_1, m_1) \longrightarrow \text{AgSt}(H, i + 1, [[\mathcal{K}_H^{i+1}]]^\mu), \text{Post}_{i+1}, \text{Snd}(H, l_2, A_2, [[m_2]]^\mu)$ 
24:           continue to next  $\pi$ 
25:         end for
26:       end if
27:     end for
28:   end for
29:   if  $\text{variantExists} = \text{false}$  then
30:     Mutate transition  $i$  to skip send:  $\text{AgSt}(H, i, \mathcal{K}_H^i), \text{Pre}_i, \text{Rcv}(H, l_1, A_1, m_1) \longrightarrow \text{AgSt}(H, i + 1, [[\mathcal{K}_H^{i+1}]]^\mu), \text{Post}_{i+1}, \text{Snd}(H, l_2, A_2, m_2)$ 
31:   end if
32: end if

```

5 Automated Analysis with the X-Men 2.0 Tool

5.1 A Brief Introduction to X-Men 2.0

X-Men [20] is a Java-based tool built on top of the Tamarin Prover [2, 14] for modeling and analyzing security ceremonies with mutations. Given a ceremony specification, X-Men applies selected human-behavior mutations and generates corresponding mutated ceremony models, which are then analyzed in Tamarin. The underlying approach is generic and tool-agnostic, and can be applied with other formal tools such as ProVerif [8] and AVANTSSAR [1].

X-Men 2.0 is a prototypical extension of X-Men to implement the *forget* mutation. Let us now discuss how the Bank Login ceremony is formally modeled and analyzed using the X-Men approach.

5.2 Formal Model of the Bank Login Ceremony

We point the reader to [2] for details on Tamarin's syntax. We already mentioned that *agent rules* represent state transitions in a ceremony. Moreover, *channel rules* model the initial communication steps between ceremony participants, corresponding to the

different channel types. Ceremony roles are processed by specifying them within the actions of a *setup* rule. *Restrictions* model constraints on ceremony rules by restricting traces, behaviors, and agent states. Due to space constraints, the complete specification of the Bank Login ceremony is shown in the appendix. Notably, the initial knowledge of the human agent comprises agent names, keys, usernames, and passwords corresponding to the two banks.

Lemmas specify the security properties to be proven and are defined over traces of action facts. In Tamarin, security properties are specified using the constructs given in the appendix (provided as supplemental material). The security goals *[SG1]* and *[SG2]* (cf. Section 3.5) are expressed by means of the Tamarin lemma in Listings 1 and 2, respectively.

```

lemma Human_intends_Bank2_if_Bank2_OK: all-traces
  ``All u b2 pw #i.
    LoginOK(b2, u, pw) @ #i
    ==> (Ex nH #j.
      U_LoginRequest(u, b2, nH) @ #j
      & #j < #i)''

```

Listing 1: Lemma formalizing [SG1].

```

lemma Password2_confidential: all-traces
  ``All u b pw nH nB #i.
    PasswordAttempt(u, b, pw, nH, nB) @ #i
  ==> not (Ex #t. K(pw) @ #t)''

```

Listing 2: Lemma formalizing [SG2].

5.3 Formal and Automated Analysis of the Bank Login Ceremony

When given our mutated specification generated by X-Men 2.0, Tamarin automatically produces an attack trace that captures the attack illustrated in Figure 2.

Let us explain this in more detail. The lemma in Listing 1 formalizes an *authentication* property: a bank $B2$ should reach a successful authentication state only if the human user H has previously initiated a login request explicitly targeting $B2$. Tamarin falsifies this lemma by producing a counterexample trace (illustrated by the MSC in Figure 2). In the trace, H initiates a login session with $B1$, which is compromised by the attacker. Although the attacker cannot decrypt some of the exchanged protocol messages, it can still forward encrypted messages unchanged. This causes a parallel execution of the login ceremony with a second bank $B2$. The attack is enabled by the *forget* mutation: when asked for credentials, H forgets the password associated with $B1$ and instead supplies the password intended for $B2$. As a result, $B2$ accepts the forwarded authentication message, which completes the ceremony and reaches a successful authentication state, although H never intended to authenticate with $B2$ in the first place.

The lemma in Listing 2 expresses a *confidentiality* property: whenever H attempts to authenticate using a password, that password is never derivable by the adversary at any point in the execution. This property is successfully verified by Tamarin for password $PW2$. Even in the scenario of bank $B1$ compromised and message forwarding, the password $PW2$ remains protected by symmetric encryption under a session key derived from fresh nonces (whereas the attacker obviously knows $PW1$ since they compromised $B1$).

6 Related Work

Several approaches [13, 19, 23] have studied the impact of human errors in security-critical systems. Unlike these works, our mutation-based approach remains effective even in the absence of an explicit attacker. Fortis [23] focuses on deviations arising from the environment, including human users, and supports repair by generating deviations under which safety properties still hold. This differs from the X-Men approach, which has been evaluated on case studies such as Oyster [21, 22]. Similarly, [19] adapted the X-Men approach to surgical procedures, using UPPAAL [4] to detect violations caused by human errors.

Tamarin Prover [2, 14] has been used to analyze the impact of threats stemming from human behavior in distributed protocols. Basin et al. [3] defined rules to capture human errors (e.g., unintended sends or receives, improper comparisons, and secret disclosure via social engineering) and verified authentication and secrecy properties for protocols such as MP-Auth and OTP-over-SMS. Bella et al. [6] used epistemic modal logic to formalize intentional and erroneous human-level threats in ceremonies, analyzing

security properties under combinations of threat models while assuming technical agents strictly follow the ceremony specification. In contrast, X-Men automatically generates human mutations with propagation rules that capture their impact on other participants' behavior, producing executable traces for vulnerability analysis; X-Men 2.0 further expands the mutation space for richer analysis.

Fila and Radomirović [11] presented a formal framework for modeling ceremonies with synchronous agent interactions, including humans. They modeled execution contexts as parallel ceremonies to capture their impact on human behavior and distinguish normal from distracted actions. Their approach, unlike ours, uses non-monotonic *availability sets* to model an agent's knowledge, allowing information to be both gained and lost over time, including forgetting via auxiliary “losing” ceremonies. While this allows occasional information loss rather than enforcing forgetting in every run, modeling non-monotonic knowledge in Tamarin enlarges the search space, since the tool must explore whether terms are present, absent, later acquired, or lost—an explosion worsened when DY attackers may learn them. Consequently, analyzing ceremonies with evolving agent knowledge becomes slow (or even infeasible).

In contrast, our approach models the *forget* mutation directly within the main ceremony. We implemented this in a structured way by defining a forget set that records forgotten messages. This preserves the (monotonic) knowledge of the human agent by locking away forgotten information and checking whether intended messages can be constructed from what remains in the agent's knowledge; forgotten messages cannot be used unless they are received again. In approach [11], forgetting may terminate the ceremony execution, preventing further analysis. In contrast, our approach allows the execution to continue after forgetting by allowing message construction via replacement or by skipping send actions, while propagating the mutation's effects. This enables analyzing vulnerabilities that arise during ongoing ceremonies. For example, we uncovered a vulnerability in the Bank Login ceremony where the human forgets the password and still attempts to conclude the ceremony using only the information they retain.

7 Conclusion and Future Work

This paper extends the X-Men approach by formalizing the *forget* mutation, a novel primitive mutation. We also implemented a prototypical extension of the X-Men tool, X-Men 2.0, which supports the integration of new mutations (e.g., *forget*). Unlike prior mutations that alter messages or ceremony actions and their ordering, *forget* restricts the *usability* of knowledge of the human agents participating in the ceremony, while preserving monotonic knowledge accumulation. We applied *forget* to a realistic Bank Login ceremony, uncovering a cross-execution credential replay attack arising from human memory lapses, despite the underlying cryptographic protocol remaining secure within the DY model.

For future work, we identify two key directions: (1) extending the approach by generalizing mutation propagation to multi-human ceremony models, and (2) investigating mutations arising from agent actions in coordinated, collaborative, or competitive settings. Overall, these directions aim to broaden the applicability of human-induced knowledge mutations and strengthen automated support for analyzing ceremony vulnerabilities.

References

- [1] Alessandro Armando, William Arsac, Tom Avanesov, Jorge Cuéllar, and Others. 2012. The AVANTSSAR Platform for the Automated Validation of Trust and Security of Service-Oriented Architectures. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2012) (Lecture Notes in Computer Science, Vol. 7214)*. Springer, Tallinn, Estonia, 267–282.
- [2] David Basin, Cas Cremers, Jannik Dreier, and Ralf Sasse. 2025. *Modeling and analyzing security protocols with Tamarin: a comprehensive guide*. Springer Nature.
- [3] David Basin, Saša Radomirović, and Lara Schmid. 2016. Modeling human errors in security protocols. In *2016 IEEE 29th Computer Security Foundations Symposium (CSF)*. IEEE, Lisbon, Portugal, 325–340.
- [4] Gerd Behrmann, Alexandre David, and Kim G Larsen. 2004. A tutorial on uppaal. *Formal methods for the design of real-time systems* 3185 (2004), 200–236.
- [5] Giampaolo Bella and Lizzie Coles-Kemp. 2012. Layered analysis of security ceremonies. In *IFIP International Information Security Conference*. Springer, Heraklion, Crete, Greece, 273–286.
- [6] Giampaolo Bella, Rosario Giustolisi, and Carsten Schürmann. 2022. Modelling human threats in security ceremonies. *Journal of Computer Security* 30, 3 (2022), 411–433.
- [7] Massimo Benerecetti and Adriano Peron. 2013. Timed protocol insecurity problem is NP-complete. *Future Generation Computer Systems* 29, 3 (2013), 843–862.
- [8] Bruno Blanchet. 2012. Automatic verification of security protocols in the symbolic model: The verifier ProVerif. In *International School on Foundations of Security Analysis and Design*. Springer, Bertinoro, Italy, 54–87.
- [9] Danny Dolev and Andrew Yao. 2003. On the security of public key protocols. *IEEE Transactions on information theory* 29, 2 (2003), 198–208.
- [10] Carl Ellison. 2007. Ceremony Design and Analysis. Cryptology ePrint Archive, Paper 2007/399. <https://eprint.iacr.org/2007/399>
- [11] Barbara Fila and Saša Radomirović. 2024. Nothing is out-of-band: formal modeling of ceremonies. In *2024 IEEE 37th Computer Security Foundations Symposium (CSF)*. IEEE, Enschede, Netherlands, 464–478.
- [12] Beate Grawemeyer and Hilary Johnson. 2011. Using and managing multiple passwords: A week to a view. *Interacting with computers* 23, 3 (2011), 256–267.
- [13] Charlie Jacomme and Steve Kremer. 2021. An extensive formal analysis of multi-factor authentication protocols. *ACM Transactions on Privacy and Security (TOPS)* 24, 2 (2021), 1–34.
- [14] Simon Meier, Benedikt Schmidt, Cas Cremers, and David Basin. 2013. The TAMARIN prover for the symbolic analysis of security protocols. In *Computer Aided Verification: 25th International Conference, CAV 2013, Saint Petersburg, Russia, July 13–19, 2013. Proceedings 25*. Springer, Saint Petersburg, Russia, 696–701.
- [15] Roger M Needham and Michael D Schroeder. 1978. Using encryption for authentication in large networks of computers. *Commun. ACM* 21, 12 (1978), 993–999.
- [16] Megha Quamara and Luca Viganò. 2025. User Identification Procedures with Human Mutations: Formal Analysis and Pilot Study. In *2025 IEEE 22nd International Conference on Software Architecture Companion (ICSA-C)*. IEEE, Odense, Denmark, 281–290.
- [17] Michaël Rusinowitch and Mathieu Turuani. 2001. *Protocol insecurity with finite number of sessions is NP-complete*. Ph. D. Dissertation. INRIA.
- [18] Michaël Rusinowitch and Mathieu Turuani. 2003. Protocol insecurity with a finite number of sessions and composed keys is NP-complete. *Theoretical Computer Science* 299, 1–3 (2003), 451–475.
- [19] Ioana Sandu, Rita Borgo, Prokar Dasgupta, Ramesh Thurairaja, and Luca Viganò. 2024. A Formal Approach for Modelling and Analysing Surgical Procedures. In *International Workshop on Security and Trust Management*. Springer, Bydgoszcz, Poland, 82–93.
- [20] Diego Sempereboni and Luca Viganò. 2020. X-Men. <https://github.com/diegosemperboni/X-Men>. Accessed: November 2024.
- [21] Diego Sempereboni and Luca Viganò. 2020. X-men: A mutation-based approach for the formal analysis of security ceremonies. In *2020 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, Genoa, Italy, 87–104.
- [22] Diego Sempereboni and Luca Viganò. 2023. A mutation-based approach for the formal and automated analysis of security ceremonies. *Journal of Computer Security* 31, 4 (2023), 293–364.
- [23] Changjian Zhang, Ian Dardik, Rômulo Meira-Góes, David Garlan, and Eunsuk Kang. 2023. Fortis: A tool for analysis and repair of robust software systems. In *2023 Formal Methods in Computer-Aided Design (FMCAD)*. IEEE, Ames, Iowa, USA, 1–9.

A Formal Model of the Bank Login Ceremony

A.1 Role Script

The role-script for bank B is shown in Figure 5.

RoleScript_B =
 $[Start(B, \langle \langle 'agentname', 'agentname', 'key', 'key', 'key' \rangle, \langle B, H, K_B, K_B^{-1}, K_H \rangle \rangle),$
 $Rcv(B, conf, H, \langle \langle 'agentname', 'agentname', 'nonce' \rangle, \langle H, B, N_H \rangle \rangle_{K_B})$
 $Snd(B, conf, H, \langle \langle 'nonce', 'nonce' \rangle, \langle N_H, N_B \rangle \rangle_{K_H}),$
 $Rcv(B, sec, H, \langle \langle 'agentname', 'password' \rangle, \langle H, PW \rangle \rangle_{N_H, N_B}),$
 $Snd(B, inse, H, \langle 'access', Granted \rangle)]$

Figure 5: Role-script for B in the Bank Login ceremony.

A.2 Agent Rules

The agent rules for the role corresponding to the human H and bank B in the Bank Login ceremony (see Figure 1) are shown in Figures 6 and 7.

(H_0) []
 $\xrightarrow{Start(H, \langle \langle 'agentname', 'agentname', 'password', 'key', 'key', 'key' \rangle, \langle H, B, PW, K_H, K_H^{-1}, K_B \rangle \rangle)}$
 $[AgSt(H, 1, \langle H, B, PW, K_H, K_H^{-1}, K_B \rangle)]$

(H_1) $[AgSt(H, 1, \langle H, B, PW, K_H, K_H^{-1}, K_B \rangle)]$
 $\xrightarrow{Snd(H, conf, B, \langle \langle 'agentname', 'agentname', 'nonce' \rangle, \langle H, B, N_H \rangle \rangle_{K_B})}$
 $[AgSt(H, 2, \langle H, B, PW, K_H, K_H^{-1}, K_B, N_H \rangle), Out_{conf}(H, B, \langle \langle 'agentname', 'agentname', 'nonce' \rangle, \langle H, B, N_H \rangle \rangle_{K_B})]$

(H_2) $[AgSt(H, 2, \langle H, B, PW, K_H, K_H^{-1}, K_B, N_H \rangle), In_{conf}(B, H, \langle \langle 'nonce', 'nonce' \rangle, \langle N_H, N_B \rangle \rangle_{K_H})]$
 $\xrightarrow{Rcv(H, conf, B, \langle \langle 'nonce', 'nonce' \rangle, \langle N_H, N_B \rangle \rangle_{K_H})}$
 $[AgSt(H, 3, \langle H, B, PW, K_H, K_H^{-1}, K_B, N_H, N_B \rangle)]$

(H_3) $[AgSt(H, 3, \langle H, B, PW, K_H, K_H^{-1}, K_B, N_H, N_B \rangle)]$
 $\xrightarrow{Snd(H, sec, B, \langle \langle 'agentname', 'password' \rangle, \langle H, PW \rangle \rangle_{N_H, N_B})}$
 $[AgSt(H, 4, \langle H, B, PW, K_H, K_H^{-1}, K_B, N_H, N_B \rangle), Out_{sec}(H, sec, B, \langle \langle 'agentname', 'password' \rangle, \langle H, PW \rangle \rangle_{N_H, N_B})]$

(H_4) $[AgSt(H, 4, \langle H, B, PW, K_H, K_H^{-1}, K_B, N_H, N_B \rangle), In_{insec}(B, H, \langle 'access', Granted \rangle)]$
 $\xrightarrow{Rcv(H, inse, B, \langle 'access', Granted \rangle)}$
 []

Figure 6: Agent rules for H in the Bank Login ceremony.

A.3 Language Constructs for defining Security Properties as Tamarin Lemmas

- Variables define, e.g., the roles of the ceremony, the messages exchanged between the roles, and the temporal ordering between the events.
- Universal quantifiers (All) and existential quantifiers (Ex) define the extent to which a property holds for the variables, with temporal variables prefixed with #.
- Logical operators; implication ($\Rightarrow$), conjunction ($\&$), disjunction ($\vee$), and negation ($\neg$).
- Timestamped action facts ($\#i @ i$, where the sort prefix for the temporal variable ' i ' is optional).

(B_0) []
 $\xrightarrow{Start(B, \langle 'agentname', 'agentname', 'key', 'key', 'key' \rangle, \langle H, B, K_B, K_B^{-1}, K_H \rangle \rangle)}$
 $[AgSt(B, 1, \langle H, B, K_B, K_B^{-1}, K_H \rangle)]$

(B_1) $[AgSt(B, 1, \langle H, B, K_B, K_B^{-1}, K_H \rangle), In_{conf}(H, B, \langle \langle 'agentname', 'agentname', 'nonce' \rangle, \langle H, B, N_H \rangle \rangle_{K_B})]$
 $\xrightarrow{Rcv(B, conf, H, \langle \langle 'agentname', 'agentname', 'nonce' \rangle, \langle H, B, N_H \rangle \rangle_{K_B})}$
 $[AgSt(B, 2, \langle H, B, K_B, K_B^{-1}, K_H, N_H \rangle)]$

(B_2) $[AgSt(B, 2, \langle H, B, K_B, K_B^{-1}, K_H, N_H \rangle)]$
 $\xrightarrow{Snd(B, conf, H, \langle \langle 'nonce', 'nonce' \rangle, \langle N_H, N_B \rangle \rangle_{K_H})}$
 $[AgSt(B, 3, \langle H, B, K_B, K_B^{-1}, K_H, N_H, N_B \rangle), Out_{conf}(B, H, \langle \langle 'nonce', 'nonce' \rangle, \langle N_H, N_B \rangle \rangle_{K_H})]$

(B_3) $[AgSt(B, 3, \langle H, B, K_B, K_B^{-1}, K_H, N_H, N_B \rangle), In_{sec}(H, B, \langle \langle 'agentname', 'password' \rangle, \langle H, PW \rangle \rangle_{N_H, N_B})]$
 $\xrightarrow{Rcv(B, sec, H, \langle \langle 'agentname', 'password' \rangle, \langle H, PW \rangle \rangle_{N_H, N_B})}$
 $[AgSt(B, 4, \langle H, B, K_B, K_B^{-1}, K_H, N_H, N_B, PW \rangle)]$

(B_4) $[AgSt(B, 4, \langle H, B, K_B, K_B^{-1}, K_H, N_H, N_B, PW \rangle)]$
 $\xrightarrow{Snd(B, inse, H, \langle 'access', Granted \rangle)}$
 $[AgSt(B, 5, \langle H, B, K_B, K_B^{-1}, K_H, N_H, N_B, PW \rangle), Out_{insec}(B, H, \langle 'access', Granted \rangle)]$

Figure 7: Agent rules for B in the Bank Login ceremony.

• (In)equalities:

- Timestamp inequality ($i < j$) for temporal ordering
- Timestamp equality ($\#i = \#j$).
- Term equality ($x = y$) for message variables ' x ' and ' y '.

A.4 Channel Rules in Tamarin

We model communication between agents using Tamarin-style channel rules. In addition to insecure channels, we use the channel types *confidential* and *secure*. A confidential channel guarantees message confidentiality, while a secure channel additionally provides agent authenticity. Channels transport messages as cipher texts rather than plain text.

Channel behavior is defined by pairs of send and receive rules. A send action SndC or SndS is transformed into a confidential action !Conf or secure action !Sec, respectively. The corresponding receive rules ChanRcv_C or ChanRcv_S consume these facts to deliver the message to the recipient. Persistent channel facts allow messages to be forwarded.

```
rule ChanSnd_C:
[ SndC(A, B, c) ]
--[ ChanSndC(A, B, c) ]->
[ !Conf(A, B, c) ]
```

Listing 3: Confidential-channel send.

```
rule ChanRcv_C:
[ !Conf(A, B, c) ]
--[ ChanRcvC(A, B, c) ]->
[ RcvC(A, B, c) ]
```

Listing 4: Confidential-channel receive.

```
rule ChanSnd_S:
[ SndS(A, B, c) ]
--[ ChanSndS(A, B, c) ]->
[ !Sec(A, B, c) ]
```

Listing 5: Secure-channel send.

```

rule ChanRcv_S:
[ !Sec(A,B,c) ]
--[ ChanRcvS(A,B,c) ]->
[ RcvS(A,B,c) ]

```

Listing 6: Secure-channel receive.**A.5 Setup Rule in Tamarin**

The rule in Listing 7 creates a fresh session challenge nH and stores the session state St_U_1 (. . .) together with the user's password. The event `SessionStart` explicitly records the beginning of a ceremony instance.

```

rule SetupSession:
[ Fr(~nH)
, !Pw1($U, pw1)
, !Pw2($U, pw2)
, !Pk($B1, pkB1)
, !Pk($B2, pkB2)
]
--[ SessionStart($U,$B1,$B2,~nH) ]->
[ St_U_1($U,$B1,$B2,~nH,pw1,pw2)
, !Type($U,'nonce',~nH)
, !Type($U,'agentname',$B1)
, !Type($U,'agentname',$B2)
]

```

Listing 7: Session setup rule.

The rules in Listings 8 and 9 initialize for each agent a fresh long-term private key and publish the corresponding public key.

```

rule RegisterUser:
[ Fr(~ltkU), Fr(~pw1), Fr(~pw2) ]
-->
[ !Ltk($U, ~ltkU)
, !Pk($U, pk(~ltkU))
, !Pw1($U, ~pw1)
, !Pw2($U, ~pw2)
, Out(pk(~ltkU))
, !Type($U,'agentname',$U)
, !Type($U,'password',~pw1)
, !Type($U,'password',~pw2)
]

```

Listing 8: Human user registration rule.

```

rule RegisterBank:
[ Fr(~ltkB) ]
-->
[ !Ltk($B, ~ltkB)
, !Pk($B, pk(~ltkB))
, Out(pk(~ltkB))
, !Type($B,'agentname',$B)
]

```

Listing 9: Bank registration rule.**A.6 Compromised Bank Rules in Tamarin**

The rule in Listing 10 models a bank compromise by leaking its long-term private key.

```

rule Compromise:
[ !Ltk(B, ltkB) ]
--[ RevLtk(B) ]->
[ Out(ltkB), !Compromised(B) ]

```

Listing 10: Bank compromise rule.

The rules in Listings 11 and 12 model a compromised agent being impersonated to relay messages (as cipher texts without decrypting them).

```

rule Fwd_Conf_B2_to_User_as_B1:
[ !Compromised(B1)
, !Conf(B2, U, c)
]
--[ FwdC(B1,B2,U)
, Neq(B1,B2)
]->
[ !Conf(B1, U, c) ]

```

Listing 11: Forwarding rule over a confidential channel.

```

rule Fwd_Sec_User_to_B2:
[ !Compromised(B1)
, !Sec(U, B1, c)
, !Pk(B2, pkB2)
]
--[ FwdS(B1,B2,U)
, Neq(B1,B2)
]->
[ !Sec(U, B2, c) ]

```

Listing 12: Forwarding rule over a secure channel.**A.7 Agent Rules in Tamarin**

User₁ sends the initial login request (Listing 13). Human-executed steps are explicitly marked using the action fact $H()$ at the start of the action fact. User₂ receives the bank challenge and replies, recording the password attempt (Listing 14). User₃ completes the ceremony (Listing 15).

```

rule User_1:
[ St_U_1($U,$B1,$B2,nH,pw1,pw2)
, !Pk($B1, pkB1)
]
--[ H()
, U_LoginRequest($U,$B1,nH)
]->
[ St_U_2($U,$B1,$B2,nH,pw1,pw2)
, SndC($U,$B1, aenc(< nH, $U >, pkB1))
]

```

Listing 13: User step 1.

```

rule User_2:
[ St_U_2($U,$B1,$B2,nH,pw1,pw2)
, !Ltk($U, ltkU)
, RcvC($B1,$U, aenc(< nH, nB >, pk(ltkU)))
]
--[ H()
, PasswordAttempt($U,$B1,pw2,nH,nB)
, TypeEv($U,'nonce',nB)
]->
[ St_U_3($U,$B1,$B2,nH,nB,pw1,pw2)
, SndS($U,$B1, senc(< $U, pw2, nH, nB >, k(nH,nB)))
]

```

Listing 14: User step 2.

```

rule User_3:
[ St_U_3($U,$B1,$B2,nH,nB,pw1,pw2)
, In(< 'access', 'Granted' >)
]
--[ HFin($U,'Granted') ]->
[ ]

```

Listing 15: User step 3.

Bank_Respond_1 processes the initial request, generates a fresh nonce nB , and returns the encrypted challenge (Listing 16). The auxiliary fact `BankKnowsNonce` ($\$B, \U, nH) records that the bank has learned nH , which is used for deriving the keys (cf. Listing 18). Bank_Respond_2 checks the secure-channel response and, on success, logs `LoginOK` and returns a session-encrypted message (Listing 17).

```
rule Bank_Respond_1:
[ !Ltk($B, ltkB)
, RcvC($U,$B, aenc(< nH, $U >, pk(ltkB)))
, !Pk($U, pkU)
, Fr(~nB)
]
--[ B_Running($B,$U,nH,~nB) ]->
[ St_B_2($B,$U,nH,~nB)
, SndC($B,$U, aenc(< nH, ~nB >, pkU))
, BankKnowsNonce($B,$U,nH)
, !Type($B,'nonce',~nB)
]
```

Listing 16: Bank response step 1.

```
rule Bank_Respond_2:
[ St_B_2($B,$U,nH,nB)
, RcvS($U,$B, senc(< $U, pw, nH, nB >, k(nH,nB)))
]
--[ LoginOK($B,$U,pw) ]->
[ St_B_3($B,$U,nH,nB)
, Out(< 'access', 'Granted' >)
, !Type($B,'access','Granted')
]
```

Listing 17: Bank response step 2.

```
rule Bank_Reuses_Nonce:
[ BankKnowsNonce(B1, U, nH)
, !Pk(B2, pkB2)
]
--[ ReusesNonce(B1,B2,nH)
, Neq(B1,B2)
]->
[ SndC(U, B2, aenc(< nH, U >, pkB2)) ]
```

Listing 18: Nonce reuse rule.

Received 25 January 2026; revised ; accepted